PayChain — Stellar Instaward: Round Two

Settlement Rule Engine, Withdraw/Redeem Flow & Stress Test — Submission Evidence

> All work described in this document runs on the **Stellar public testnet** with a **simulated** M-PESA B2C disbursement leg. No real money moves and no NCBA banking rail is touched at any point. Access is restricted to flagged Stellar pilot (`isDemoMerchant`) accounts; the real PayChain merchant dashboard and mobile app are completely unchanged.

Pilot Merchants — Closed-Beta Roster

Five realistic (invented) Kenyan SMEs, each a flagged demo account with its own Stellar testnet wallet and a pre-configured settlement rule, created via `backend/scripts/create-stellar-pilot-merchants.js`. They sign in on the demo app exactly like a real merchant would (password only — OTP is intentionally skipped for demo/pilot accounts). One account (Lakeside Hardware) is a deliberate **control** — its settlement rule is switched off, to prove the engine never touches a merchant who hasn't opted in.

Merchant	County	Business Type	Settlement Rule
Wanjiku Fresh Greens Ltd	Nairobi (Kawangware)	Limited Liability Co.	20% to Stellar — float stays mostly liquid
Kilimani Brew House	Nairobi (Kilimani)	Limited Liability Co.	50% to Stellar
Pwani Fresh Fish Traders	Mombasa (Nyali)	Partnership	80% to Stellar — reserve-heavy
Rift Valley Dairy Cooperative	Nakuru	Cooperative Society	20% to Stellar
Lakeside Hardware & Building Supplies	Kisumu	Sole Proprietorship	Off — control account

Named Pilot Merchant Activity

Real testnet activity generated against these specific named accounts (distinct from the stress test's throwaway local accounts further below) via `backend/scripts/generate-pilot-demo-activity.js`. Lakeside Hardware correctly produced no settlement entry — the control account's rule is off.

Merchant	Kind	USDC	Testnet Transaction (StellarExpert)
Wanjiku Fresh Greens Ltd	split settlement	3.8461538	42199ec5e82bc3…8b991cfcde
Wanjiku Fresh Greens Ltd	redemption	3	c0c0b3b87ef8e5…57d4a578c2
Kilimani Brew House	split settlement	18.4615385	7824911b30f440…15d9cd8ebe
Kilimani Brew House	redemption	10	3bedad6c3e39ba…1539220760
Pwani Fresh Fish Traders	split settlement	38.1538462	0e05327547b8d4…878161bb29
Rift Valley Dairy Cooperative	split settlement	4.7692308	066804e61ff103…243eeaefd6

PayChain — Stellar Instaward: Round Two

Settlement Rule Engine, Withdraw/Redeem Flow & Stress Test — Submission Evidence

Deliverable 1 — Settlement Rule Configuration

Requirement	Implementation
Percentage-split fields on the merchant schema	backend/models/Merchant.js → settlementRule { enabled, stellarSharePercent }
Backend calculates the split on incoming payments	stellarSettlementService.js (computeSplit, applySettlementSplit), hooked after the KES credit in ncbaLedgerService.js (creditNcbaCollection) and both STK settlement paths in mpesaController.js
Slider + save in the merchant UI	apps/demo/src/components/stellar/SettlementRuleCard.jsx (Inflation Shield page)
API	GET / PUT /api/transactions/settlement-rule

Design notes: the split runs after the KES credit, detached, so a slow Stellar network can never delay or break a payment notification. A unique index makes a webhook retry a no-op — exactly one split per incoming payment. Non-pilot (real) merchants are never affected by any of this.

Deliverable 2 — Withdraw / Redeem

Requirement	Implementation
Verify balance, sign a Stellar debit, generate a receipt hash	redeemToMpesa — on-chain balance check, swapUsdcToKesOnChain, SHA-256 receipt hash over canonical JSON
Simulated Safaricom B2C payload/response	buildSimulatedB2C — Daraja-shaped payload; callback URLs on the reserved .invalid TLD so nothing can ever actually be sent
“Request Payout” modal, loading/success states	apps/demo/src/components/stellar/RequestPayoutModal.jsx
Reconciliation table linked to StellarExpert + download	ReconciliationTable.jsx · GET /api/transactions/wallet/reconciliation(.csv)
API	POST /api/transactions/wallet/redeem

Error Handling — Verified in the Stress Test

- **Low balance** → rejected as INSUFFICIENT_USDC; nothing moves.
- **Stellar RPC timeout** → the ledger entry is marked *unconfirmed*; KES is deliberately not auto-refunded (the transaction may still land on-chain, and refunding could pay the merchant twice). It stays visible for manual reconciliation.
- **Definitive rejection** (e.g. the master wallet is underfunded, or has no trustline) → marked *failed* and the KES is returned to the merchant automatically.
- **Duplicate webhook replay** → rejected by a unique index. **Double-click on Request Payout** → rejected by a 20-second guard.

Stress Test

`backend/scripts/stellar-stress-test.js` — 50 operations across 5 pilot merchants (one control with the rule off). The script refuses to run against anything but a local database.

Operations Run	Confirmed Testnet Transactions	Invariant Failures
50	**36**	**0**

Invariants checked after the run: KES conservation per merchant, on-chain USDC equals the ledger for every wallet, every receipt hash present and recomputable, the control merchant never settles, duplicate webhooks are rejected, and the exported CSV contains valid StellarExpert links. Unit tests: `backend/__tests__/stellarSettlement.test.js`.

Full Operation Log (50 Operations)

#	Operation	Pilot	Outcome
1	Payment (split)	1	Completed — 20% = KES 300 → 2.3076923 USDC
2	Payment (split)	2	Completed — 50% = KES 1,600 → 12.3076923 USDC
3	Payment (split)	3	Completed — 80% = KES 6,240 → 48 USDC
4	Payment (split)	4	Completed — 20% = KES 2,400 → 18.4615385 USDC
5	Payment (rule off)	5	Net KES 950 kept liquid
6	Payment (split)	1	Completed — 20% = KES 640 → 4.9230769 USDC
7	Payment (split)	2	Completed — 50% = KES 3,900 → 30 USDC
8	Payment (split)	3	Completed — 80% = KES 9,600 → 73.8461538 USDC
9	Payment (split)	4	Completed — 20% = KES 190 → 1.4615385 USDC
10	Payment (rule off)	5	Net KES 20,000 kept liquid
11	Payment (split)	1	Completed — 20% = KES 1,560 → 12 USDC
12	Payment (split)	2	Completed — 50% = KES 6,000 → 46.1538462 USDC
13	Payment + RPC timeout	3	Handled: unconfirmed
14	Payment (split)	4	Completed — 20% = KES 4,000 → 30.7692308 USDC
15	Payment (rule off)	5	Net KES 1,500 kept liquid
16	Payment (split)	1	Completed — 20% = KES 2,400 → 18.4615385 USDC
17	Payment (split)	2	Completed — 50% = KES 475 → 3.6538462 USDC
18	Payment (split)	3	Completed — 80% = KES 16,000 → 123.0769231 USDC
19	Payment (split)	4	Completed — 20% = KES 300 → 2.3076923 USDC
20	Payment (rule off)	5	Net KES 3,200 kept liquid

#	Operation	Pilot	Outcome
21	Payment (split)	1	Completed — 20% = KES 190 → 1.4615385 USDC
22	Payment + master underfunded	2	Handled: failed, KES refunded
23	Payment (split)	3	Completed — 80% = KES 1,200 → 9.2307692 USDC
24	Payment (split)	4	Completed — 20% = KES 640 → 4.9230769 USDC
25	Payment (rule off)	5	Net KES 7,800 kept liquid
26	Payment (split)	1	Completed — 20% = KES 4,000 → 30.7692308 USDC
27	Payment (split)	2	Completed — 50% = KES 750 → 5.7692308 USDC
28	Payment (split)	3	Completed — 80% = KES 2,560 → 19.6923077 USDC
29	Payment (split)	4	Completed — 20% = KES 1,560 → 12 USDC
30	Payment (rule off)	5	Net KES 12,000 kept liquid
31	Redeem → simulated M-Pesa	1	KES 2,272.50 to 0701***000
32	Redeem → simulated M-Pesa	1	KES 1,704.38 to 0701***000
33	Redeem → simulated M-Pesa	1	KES 1,278.28 to 0701***000
34	Redeem → simulated M-Pesa	2	KES 3,181.26 to 0701***001
35	Redeem → simulated M-Pesa	2	KES 2,385.94 to 0701***001
36	Redeem → simulated M-Pesa	2	KES 1,789.45 to 0701***001
37	Redeem → simulated M-Pesa	3	KES 8,899.99 to 0701***002
38	Redeem → simulated M-Pesa	3	KES 6,675.01 to 0701***002
39	Redeem → simulated M-Pesa	3	KES 5,006.25 to 0701***002
40	Redeem → simulated M-Pesa	4	KES 2,272.50 to 0701***003
41	Redeem → simulated M-Pesa	4	KES 1,704.38 to 0701***003
42	Redeem → simulated M-Pesa	4	KES 1,278.28 to 0701***003
43	Redeem (no USDC)	5	Rejected: INSUFFICIENT_USDC

#	Operation	Pilot	Outcome
44	Redeem (no USDC)	5	Rejected: INSUFFICIENT_USDC
45	Redeem (no USDC)	5	Rejected: INSUFFICIENT_USDC
46	Redeem over balance	1	Rejected: INSUFFICIENT_USDC
47	Redeem over balance	2	Rejected: INSUFFICIENT_USDC
48	Redeem + RPC timeout	4	Handled: RPC_TIMEOUT
49	Redeem → simulated M-Pesa	1	KES 383.49 to 0701***000
50	Redeem → simulated M-Pesa	3	KES 1,501.88 to 0701***002

Transaction Hashes — Live Testnet Run (Stress Test)

36 confirmed Stellar testnet transactions from the stress test's own throwaway local accounts (separate from the named pilots above). Each hash is a clickable link to its StellarExpert testnet explorer page.

Pilot	Kind	USDC	Testnet Transaction (StellarExpert)
Pilot 1	split settlement	2.3076923	11d70afef55258…dc034c5d4c
Pilot 1	split settlement	4.9230769	a206f115e7a9f4…deee2f6174
Pilot 1	split settlement	12	8b3724ad23e203…10d4610467
Pilot 1	split settlement	18.4615385	3ea3deef3bd9b7…4c3c21997e
Pilot 1	split settlement	1.4615385	94005b6918b823…952ddcab74
Pilot 1	split settlement	30.7692308	63f7a6e677e998…256a9abeb5
Pilot 1	redemption	17.4808	7e8d9fdac15be8…6056e2e1e5
Pilot 1	redemption	13.1106	ed0681089eb070…d4631b06bc
Pilot 1	redemption	9.8329	881f361abb3762…3ca17f2f66
Pilot 1	redemption	2.9499	d432cd7134c0b3…a42c77b4ce
Pilot 2	split settlement	12.3076923	96e98e6e5314bc…47efbe0926
Pilot 2	split settlement	30	235527ea030dc3…0ff876bc73
Pilot 2	split settlement	46.1538462	b3c54fc1ff2937…c9a305cd4f
Pilot 2	split settlement	3.6538462	3d745e690ad26e…096bed9476
Pilot 2	split settlement	5.7692308	af05a92f8a89f9…15e960b8b1
Pilot 2	redemption	24.4712	51580f83c7bcdc…149a5890c0
Pilot 2	redemption	18.3534	6a9acf7d78cbb5…df20eeb22f
Pilot 2	redemption	13.765	6635aa37672b1b…ea68c35c86
Pilot 3	split settlement	48	b42a1572b22b04…af9531a7f8
Pilot 3	split settlement	73.8461538	bc931fbbcffdaa…c398007f81
Pilot 3	split settlement	123.0769231	89984a460f78be…1b582d9667
Pilot 3	split settlement	9.2307692	c3bbb42b5d208f…811ceafa9b
Pilot 3	split settlement	19.6923077	81fc8a80f4332e…ced71111a7

Pilot	Kind	USDC	Testnet Transaction (StellarExpert)
Pilot 3	redemption	68.4615	233ada0c788675…78d5a82d2e
Pilot 3	redemption	51.3462	fc0d75ba08facd…b4da1a687a
Pilot 3	redemption	38.5096	5af7feac19d308…9d114645c3
Pilot 3	redemption	11.5529	acfbc7673e5215…7a5758ba5f
Pilot 4	split settlement	18.4615385	c3c5231728a5f2…811865baa7
Pilot 4	split settlement	1.4615385	1202750ce45f49…a6170775fd
Pilot 4	split settlement	30.7692308	ff2a33a51887f2…402a79e09e
Pilot 4	split settlement	2.3076923	8de2bf0d4e76e4…186be38da8
Pilot 4	split settlement	4.9230769	2b8f85e88b3295…19b15adc35
Pilot 4	split settlement	12	1e130c7adf99d8…71e3d2a137
Pilot 4	redemption	17.4808	9dc0202f635b91…7895a08bdd
Pilot 4	redemption	13.1106	0e4c5a074eb71b…f00b750f41
Pilot 4	redemption	9.8329	395ab4c2747451…f497315623

Demo Video — Walkthrough Script

Recorded against **Wanjiku Fresh Greens Ltd** (paychain58@gmail.com), which already has a real split settlement and a real redemption on testnet to point at — no live transaction needs to succeed on camera. Total run time ~4–5 minutes.

1. Intro (15–20s)
"This is PayChain's Stellar settlement engine — Round Two of our Instaward. It lets a Kenyan merchant automatically route a percentage of every M-Pesa payment they receive into a Stellar testnet wallet as USDC, and withdraw it back to M-Pesa on demand. I'll walk through both directions using one of our real pilot merchants, Wanjiku Fresh Greens Ltd."

2. Sign in (20s)
Demo app → Digital Wallet (or Inflation Shield) → "Connect the demo Stellar wallet" → paychain58@gmail.com / SomeStrongPassword123!. "This logs in a real backend-connected merchant account — no OTP needed for our test pilots. Everything from here is real Stellar testnet activity, not a mockup."

3. Settlement rule (30–40s)
Digital Wallet → Inflation Shield → point at the Settlement Rule card. "Wanjiku has set 20% — so for every payment they receive, 80% stays liquid for daily operations, and 20% automatically converts to USDC on Stellar as an inflation hedge. This is merchant-configurable and saved per account."

4. Real settlement history (40–50s)
Scroll to the reconciliation table, point at the 3.8461538 USDC split-settlement entry. "This isn't simulated data — this is a real transaction." Click the hash, let StellarExpert load on screen (42199ec5e8…b991cfcde). "Anyone can independently verify this happened, on a public testnet explorer."

5. Withdraw / Redeem (60–90s)
Click "Request Payout." "Now the reverse direction — pulling USDC back out to M-Pesa on demand." Show loading then success. "Behind the scenes this verifies the on-chain balance, signs a real debit transaction on Wanjiku's Stellar wallet, generates a receipt hash, and triggers a simulated M-Pesa B2C payout — simulated because this is testnet, so no real money moves, but the Stellar side is 100% real." Point at the 3 USDC redemption entry, click its hash too (c0c0b3b87e…d4a578c2).

6. Wrap-up (15–20s)
"That's the full loop — M-Pesa in, automatic Stellar settlement, and withdrawal back to M-Pesa on demand, fully verifiable on a public testnet explorer. This same flow ran under stress testing across five pilot merchants with 50 operations and 36 confirmed transactions, documented in our submission evidence."

GitHub reference: `github.com/PayChainKE/paychainkeHQ · Pull Request #299`